

✓ AI Hardware Architecture

Author: Nitin Sarangdhar
Affiliation: SemiX Visiting Professor of practice | IIT Bombay
Date: March 2026

Designing a 2-Layer CNN for MNIST Classification

✓ Objective

To architect, implement, and analyze a 2-layer Convolutional Neural Network (CNN) using **PyTorch**. This lab transitions from basic edge detection to multi-class digit recognition (0–9), with a specific focus on **hardware-software co-design** and the trade-offs of deep learning silicon.

1. Architecture Specification

Build a network with the following pipeline. Pay close attention to the **SRAM footprint** and **MAC (Multiply-Accumulate)** requirements of each stage.

Layer	Type	Configuration	Hardware Role
Input	Grayscale	1 Channel, 28x28 pixels	Raw Data Stream
Conv 1	Convolution	32 Filters, 3x3 Kernel, Bias=True	Low-level feature extraction (Edges)
ReLU 1	Activation	Element-wise $\max(0, x)$	Non-linear "Feature Cleaning"
Pool 1	Max Pooling	2x2 Window, Stride 2	Spatial Compression (28x28 $\rightarrow$ 14x14)
Conv 2	Convolution	64 Filters, 3x3 Kernel, Bias=True	Mid-level feature extraction (Shapes)
ReLU 2	Activation	Element-wise $\max(0, x)$	Introducing Sparsity
Pool 2	Max Pooling	2x2 Window, Stride 2	Spatial Compression (14x14 $\rightarrow$ 7x7)
Flatten	Reshape	1D Vector (64 $\times$ 5 $\times$ 5)	Memory alignment for Dense layer
FC 1	Fully Connected	128 Neurons	High-level reasoning
FC 2	Output Layer	10 Neurons (Digits 0–9)	Final Class Scoring (Logits)

2. The Dual Role of ReLU in Silicon

In this assignment, we analyze how the mathematical ReLU function ($y = \max(0, x)$) serves two distinct architectural purposes depending on its depth in the pipeline:

Stage	Functional Role	Architectural Impact
ReLU 1	Feature Cleaning	Acts as a "noise gate" for raw pixels. It removes negative gradients and "background hiss," ensuring the next layer rece
ReLU 2	Sparsity Generation	Facilitates deep-layer selectivity . It forces the network to be "sparse" (mostly zeros), allowing sparsity-aware hardwar

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torchvision import datasets, transforms
5 from torch.utils.data import DataLoader
6
7 # --- 1. HARDWARE-AWARE ARCHITECTURE ---
8 class MNISTHardwareModel(nn.Module):
9     def __init__(self):
10         super(MNISTHardwareModel, self).__init__()
```

```

11
12     # Layer 1: Feature Cleaning (32 Filters, 3x3)
13     # Input: 28x28 -> Conv(3x3): 26x26 -> Pool(2x2): 13x13
14     self.conv1 = nn.Conv2d(1, 32, kernel_size=3, bias=True)
15
16     # Layer 2: Sparsity Generation (64 Filters, 3x3)
17     # Input: 13x13 -> Conv(3x3): 11x11 -> Pool(2x2): 5x5
18     self.conv2 = nn.Conv2d(32, 64, kernel_size=3, bias=True)
19
20     self.pool = nn.MaxPool2d(2, 2)
21
22     # Classifier (Vector-Matrix Multiplication)
23     # Flattened Input: 64 filters * 5px * 5px = 1600 elements
24     self.fc1 = nn.Linear(64 * 5 * 5, 128)
25     self.fc2 = nn.Linear(128, 10)
26
27     def forward(self, x):
28         # Stage 1: Conv -> ReLU (Clean) -> Pool
29         x = self.pool(torch.relu(self.conv1(x)))
30
31         # Stage 2: Conv -> ReLU (Sparse) -> Pool
32         x = self.pool(torch.relu(self.conv2(x)))
33
34         # Bridge: Flatten 2D Feature Maps to 1D Vector
35         x = torch.flatten(x, 1)
36
37         # Stage 3: Dense Reasoning
38         x = torch.relu(self.fc1(x))
39         x = self.fc2(x)
40         return x
41
42 # --- 2. DATA PREPARATION ---
43 transform = transforms.Compose([
44     transforms.ToTensor(),
45     transforms.Normalize((0.1307,), (0.3081,)) # Standard MNIST mean/std
46 ])
47
48 train_set = datasets.MNIST('./data', train=True, download=True, transform=transform)
49 train_loader = DataLoader(train_set, batch_size=64, shuffle=True)
50
51 # --- 3. TRAINING SETUP ---
52 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
53 model = MNISTHardwareModel().to(device)
54 optimizer = optim.Adam(model.parameters(), lr=0.001)
55 criterion = nn.CrossEntropyLoss()
56
57 print(f"Device: {device}")
58 print(f"Total Parameters: {sum(p.numel() for p in model.parameters())},")
59
60 # --- 4. MINI-TRAINING LOOP (3 Epochs) ---
61 model.train()
62 for epoch in range(1, 4):
63     running_loss = 0.0
64     for batch_idx, (data, target) in enumerate(train_loader):
65         data, target = data.to(device), target.to(device)
66
67         optimizer.zero_grad() # Reset Gradients
68         output = model(data) # Forward Pass
69         loss = criterion(output, target) # Calculate Error
70         loss.backward() # Backpropagation (Using grad_fn)
71         optimizer.step() # Update Weights (SGD/Adam)
72
73     running_loss += loss.item()
74     if batch_idx % 200 == 199:

```

```

75         print(f"Epoch {epoch} | Batch {batch_idx+1} | Loss: {running_loss/200:.4f}")
76         running_loss = 0.0
77
78 print("\nTraining Complete. The model is now an optimized digit classifier!")

```

```

100%|██████████| 9.91M/9.91M [00:01<00:00, 5.66MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 134kB/s]
100%|██████████| 1.65M/1.65M [00:01<00:00, 1.27MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 8.68MB/s]

```

Device: cuda

Total Parameters: 225,034

```

Epoch 1 | Batch 200 | Loss: 0.3579
Epoch 1 | Batch 400 | Loss: 0.1013
Epoch 1 | Batch 600 | Loss: 0.0766
Epoch 1 | Batch 800 | Loss: 0.0626
Epoch 2 | Batch 200 | Loss: 0.0437
Epoch 2 | Batch 400 | Loss: 0.0419
Epoch 2 | Batch 600 | Loss: 0.0417
Epoch 2 | Batch 800 | Loss: 0.0455
Epoch 3 | Batch 200 | Loss: 0.0287
Epoch 3 | Batch 400 | Loss: 0.0311
Epoch 3 | Batch 600 | Loss: 0.0316
Epoch 3 | Batch 800 | Loss: 0.0271

```

Training Complete. The model is now an optimized digit classifier!

```

1 import matplotlib.pyplot as plt
2 import torch
3
4 # --- 1. PREPARE A SAMPLE IMAGE ---
5 # Get one batch from the train_loader we defined earlier
6 data_iter = iter(train_loader)
7 images, labels = next(data_iter)
8
9 # Select a single image (e.g., the first one in the batch)
10 test_image = images[0].unsqueeze(0).to(device)
11 true_label = labels[0].item()
12
13 # --- 2. EXTRACT INTERMEDIATE ACTIVATIONS ---
14 # We run the forward pass manually to 'catch' the output after ReLU2
15 model.eval() # Set to evaluation mode
16 with torch.no_grad():
17     # Step-by-step pass
18     x_conv1 = torch.relu(model.conv1(test_image))
19     x_pool1 = model.pool(x_conv1)
20
21     x_conv2 = torch.relu(model.conv2(x_pool1)) # This is the ReLU2 stage
22
23     # Final prediction for context
24     output = model(test_image)
25     prediction = torch.argmax(output, dim=1).item()
26
27 # --- 3. VISUALIZATION ---
28 print(f"Target Digit: {true_label} | Model Prediction: {prediction}")
29
30 fig, axes = plt.subplots(1, 5, figsize=(15, 3))
31
32 # Plot 1: Original Image
33 axes[0].imshow(test_image.cpu().squeeze(), cmap='gray')
34 axes[0].set_title("Original (28x28)")
35
36 # Plot 2-5: Selection of 4 Filters from ReLU2
37 # We have 64 filters; let's look at a few to see the 'Sparsity'
38 for i in range(4):
39     filter_map = x_conv2[0, i].cpu().numpy()

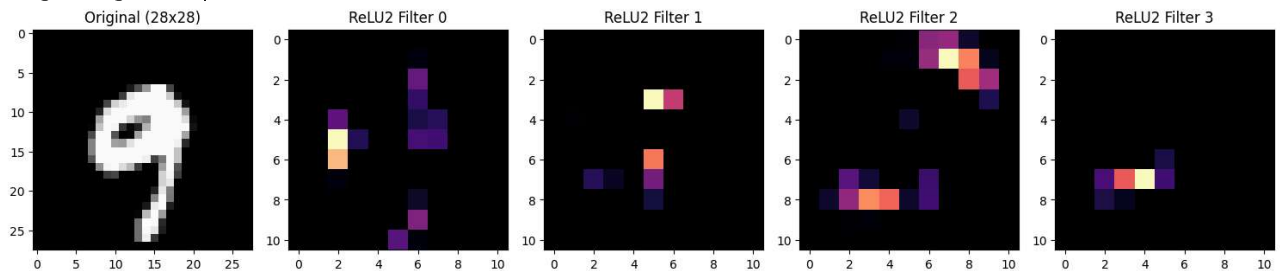
```

```

40 axes[i+1].imshow(filter_map, cmap='magma')
41 axes[i+1].set_title(f"ReLU2 Filter {i}")
42
43 plt.tight_layout()
44 plt.show()
45
46 # --- 4. SPARSITY CHECK (The Hardware Metric) ---
47 total_elements = x_conv2.numel()
48 zero_elements = (x_conv2 == 0).sum().item()
49 sparsity_percentage = (zero_elements / total_elements) * 100
50
51 print(f"--- Hardware Sparsity Report ---")
52 print(f"Total Pixels in ReLU2 Layer: {total_elements}")
53 print(f"Zero Pixels (Inactive): {zero_elements}")
54 print(f"Sparsity Level: {sparsity_percentage:.2f}%")

```

Target Digit: 9 | Model Prediction: 9



```

--- Hardware Sparsity Report ---
Total Pixels in ReLU2 Layer: 7744
Zero Pixels (Inactive): 7072
Sparsity Level: 91.32%

```

Double-click (or enter) to edit

Assignment Section: Hardware Resource Analysis Task: For each layer of the MNISTHardwareModel, calculate the total memory required to store parameters and the total number of arithmetic operations (MACs) required for a single inference pass.

Hardware Parallelization & The "Batching" Effect Objective: To observe how the hardware processes multiple images simultaneously to maximize the utilization of parallel Arithmetic Logic Units (ALUs).

The Concept: Serial vs. Parallel Throughput In your code, we use a batch_size=64. From a hardware perspective, this means the NPU/GPU is not processing one digit, then the next. Instead, it is loading the same weight tensors once and applying them to 64 different image tensors in parallel.

